FSStat

Assignment #02 — Concurrent and Distributed Programming

Eduard Toni Alexandru

a.y. 2025–2026 · Java 21, Vert.x 5.0.10, RxJava 3.1.8

1 Analysis

1.1 Requirements

Id	Requirement
R1	`getFSReport(D, MaxFS, NB)` is *asynchronous*: the caller is not blocked and obtains the report `R` when it is ready.
R2	`R` contains the total number of files in `D`, recursively.
R3	`R` contains the distribution of file sizes over $NB + 1$ bands: NB equal intervals of $[0, MaxFS]$ and one band for files bigger than `MaxFS`.
R4	An example program shows the use of the library (one per version).
R5 (ext.)	The generation of a report can be *stopped*.
R6 (ext.)	Partial statistics are *dynamically* available while the report is being generated, e.g. to show them in a GUI with start/stop buttons.
R7	Each version follows the discipline of its approach (event loop, Rx, virtual threads) and does not reuse code when this would compromise that discipline.

Table 1: Requirements (from the assignment text).

Some points left open by the text were fixed as follows. The band of a file of size s is

$$band(s) = \begin{cases} NB & s > MaxFS \\ \min\left(\lfloor s \cdot NB / MaxFS \rfloor,\ NB - 1\right) & \text{otherwise} \end{cases}$$

so a file of exactly `MaxFS` bytes falls in the last regular band (the interval is closed) and no truncation of the band width occurs when `MaxFS` is not a multiple of `NB`. Only regular files are counted; symbolic links are not followed (otherwise a link to an ancestor would create an infinite tree); unreadable files and directories are skipped, because a single permission error must not abort a report on a whole file system. An invalid root or invalid parameters make the whole request fail.

1.2 Concurrency aspects

- **I/O-bound work of unknown size.** Almost all the time is spent waiting for the file system. Concurrency is useful to have many I/O requests in flight, not to use many cores. The shape and size of the tree are discovered only while exploring it, so the number of tasks is not known in advance and new work is generated dynamically.
- **Termination detection.** The report is complete only when every directory at any depth has been processed. With dynamically generated work, completion has to be propagated bottom-up: a directory is done when all its entries are done.

- **Aggregation and shared state.** Partial reports can be merged in any order (merging just sums the counters band by band), so the base problem can be solved with *no shared mutable state*: each sub-tree returns its own report and the parent merges them. Shared state is needed only by the extension (R6), because someone must be able to observe the partial result *while* it is being built.
- **Observer vs. workers** (R6). Partial results are written by the scanning activities and read by an observer: the two must not race (no torn or inconsistent snapshots), and the observer must not be flooded (one update per file would saturate the GUI: a few updates per second are enough).
- **Cancellation** (R5). A stop request is asynchronous with respect to the scan: it must reach activities that are already running, stop the generation of new work, and leave the system in a consistent state: a final notification is delivered, no update arrives after it, and a new scan can be started without interference from the old one.
- **GUI thread confinement.** Swing components may be touched only by the Event Dispatch Thread (EDT). Results produced by library threads must be handed over to the EDT, and the EDT must never block waiting for the library.
- **Bounding resources.** Every listing and every attribute read is a pending I/O operation; exploring all sub-trees at once is possible, but each approach has its own way to absorb or limit this (worker pool, schedulers, virtual threads).

1.3 Actors and interactions

Abstracting from the technology, the interactive application involves three kinds of actors:

- the **user / GUI** (EDT), which issues *start* and *stop* commands and displays reports;
- the **scan**, a dynamic set of activities exploring the tree and producing partial results;
- an **update source**, which periodically turns the current partial result into an update for the GUI (this is what limits the update rate).

The *start* command creates a new scan; *stop* and the natural end of the scan both lead to a single final notification (*completed*, *stopped* or *failed*). Each version maps these actors onto its own concepts; the mapping is the core of the design (Section 2).

2 Design

2.1 Overall architecture and shared parts

The code is organised in three layers (Figure 1):

- `lib.*`: the three versions of the library, one package each;
- `app.*`: for each version, an example program for the base library (`...CLI`) and a GUI program for the extension (`...GUI`), with a version-specific controller;
- `view`: a Swing GUI (`FSStatView`) shared by the three GUI programs.

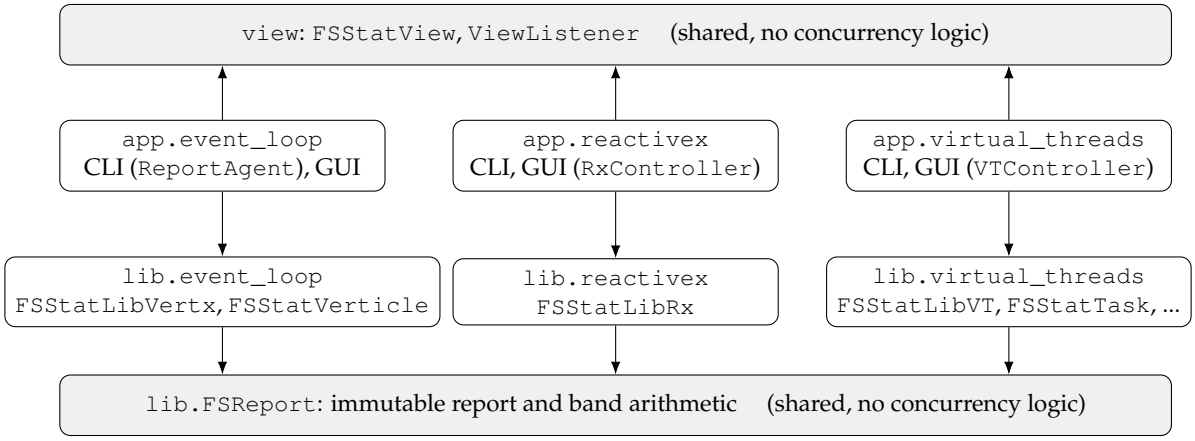


Figure 1: Packages and dependencies. Grey parts are shared by the three versions.

For each version the design is described from three points of view: a UML *class diagram* (static structure: classes, fields, methods and relations), a *runtime diagram* (which thread or event loop runs each component and how they communicate) and a UML *sequence diagram* of the interactive use case (start, periodic updates, stop). The class diagram of the shared part is in Figure 2.

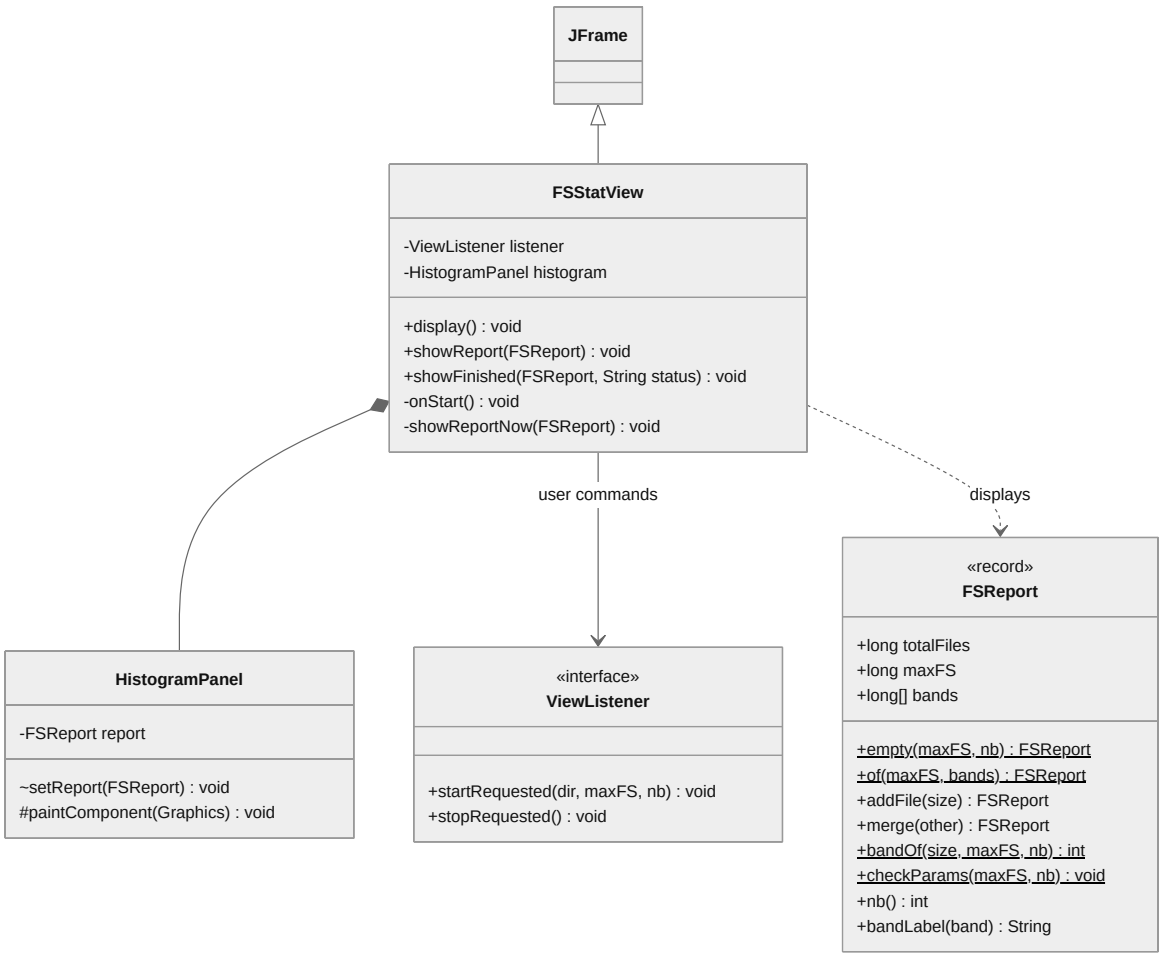


Figure 2: Class diagram of the shared part: the report value and the Swing view.

Only two parts are shared, and both are free of concurrency logic, so sharing them does not affect the discipline of any version (R7):

- `FSReport`, the immutable report with the band arithmetic (`bandOf`, `addFile`, `merge`): every version needs exactly the same arithmetic, and an immutable value can be passed between threads safely;
- the Swing view, built as in the MVC examples of the course: it forwards user commands to a `ViewListener` (the controller) and exposes `showReport` and `showFinished`, which can be called from any thread because they only schedule the update on the EDT with `SwingUtilities.invokeLater`.

Conversely, there is deliberately *no common library interface*. A single interface would force the same shape (e.g. a callback-based method) on all versions, while each approach has its own abstraction for "a result that will be available later": a Vert.x `Future`, an Rx `Single`, a blocking `Future.get()` on a virtual thread. The same holds for the extension: an event-bus protocol, a `Flowable` of partial reports, a listener called by virtual threads (Table 2). For the same reason each version has its own controller.

	Base (R1–R3)	**Extension (R5, R6)**
Event loop	`Future<FSReport> getFSReport(...)`, to be called from a verticle	`FSStatVerticle`: updates and outcome published on the event bus; stop = a message
Reactive	`Single<FSReport> getFSReport(...)`	`Flowable<FSReport> getFSReportStream(...)`: stream of partial reports; stop = `dispose()`
Virtual threads	`Future<FSReport> getFSReport(...)` computed by a virtual thread (plus the synchronous `computeFSReport`)	`FSStatTask`: an `FSStatListener` is notified (`onUpdate`, `onCompleted`, `onStopped`, `onFailed`); cooperative `stop()`

Table 2: API of the three versions.

2.2 Event-loop version: components

The event-loop version is split in two groups of classes: the base library with its CLI example (Figure 3) and the interactive extension with its GUI program (Figure 4). All the classes that run code reacting to events extend `VerticleBase`; the controller is the only class running on the Swing thread, and it interacts with the verticles only by deploying them and publishing messages.

In Vert.x the unit of concurrency is the *verticle*: an event loop that owns its state and reacts to events (I/O completions, timers, messages) one at a time. Two rules shape the design: (i) the continuations of a Vert.x future run on the event loop (context) of whoever called the asynchronous operation; (ii) different verticles never share state, they communicate through the event bus. Hence every piece of logic is placed *inside* a verticle.

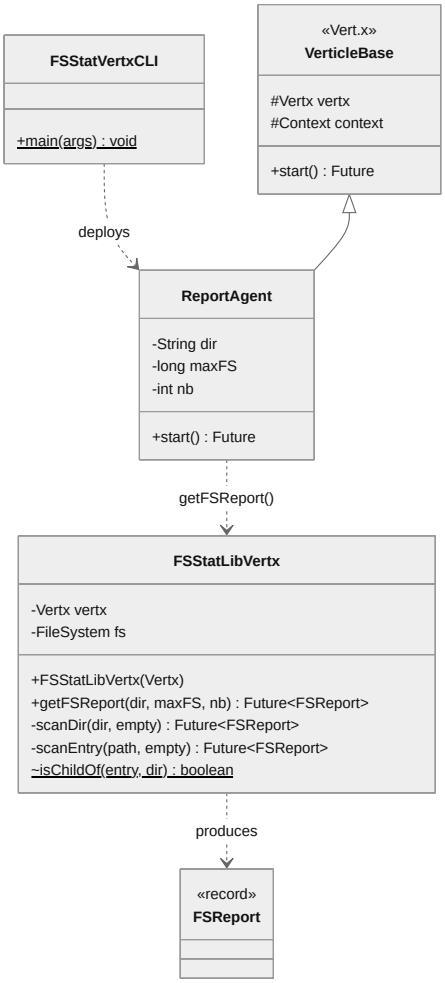


Figure 3: Event-loop version, base library and CLI example (class diagram).

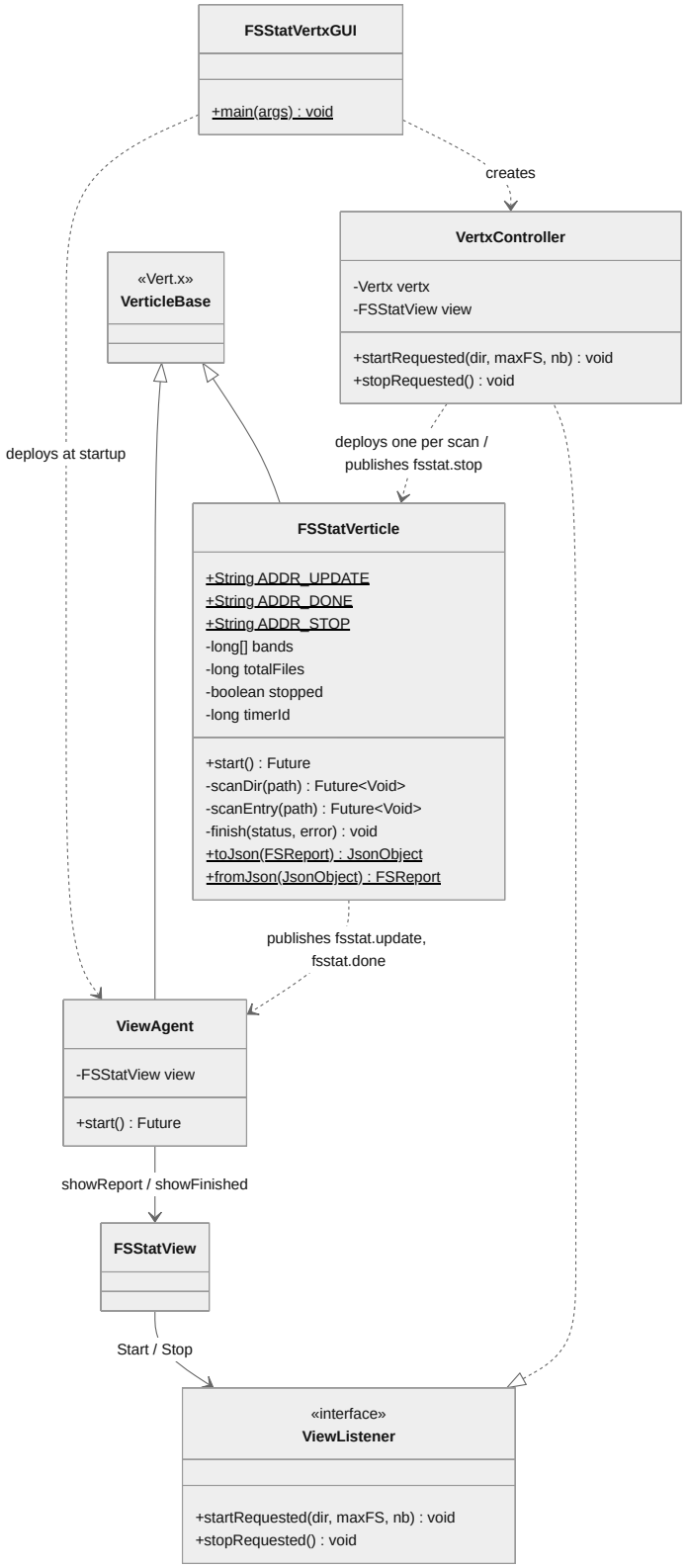


Figure 4: Event-loop version, interactive extension and GUI program (class diagram). Dashed arrows labelled with an address are event-bus messages.

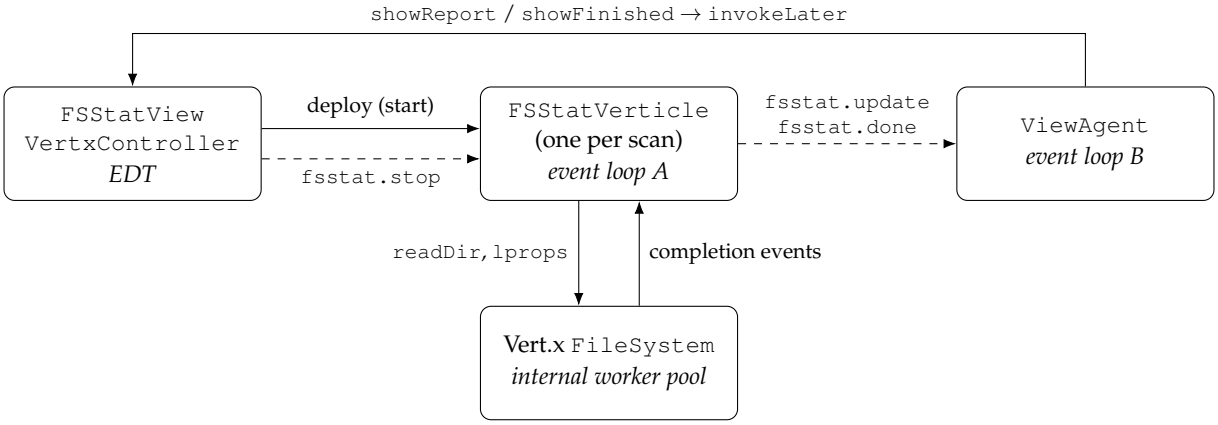


Figure 5: Event-loop version, GUI program. Dashed arrows are event-bus messages.

- **`FSStatLibVertx`** (base library) is a passive object: `getFSReport` builds a composition of futures and returns immediately. Since continuations run on the caller's event loop, the whole scan is executed by one thread and needs no synchronization, but only if the method is called *from an event loop*.
- **`ReportAgent`** (CLI example) exists for exactly that reason. The `main` thread is not an event loop: if it called the library directly, the operations would be bound to contexts that Vert.x creates implicitly for a foreign thread, and the single-thread guarantee would rest on an implementation detail rather than on the design. So `main` only creates Vert.x and deploys `ReportAgent`, a verticle whose `start()` calls `getFSReport` and prints the result when the future completes, as in the lab examples where all the logic lives in a verticle. When the report is ready the agent closes Vert.x (scheduled with `runOnContext`, so that it happens after `start()` has returned, also when the future fails immediately).
- **`FSStatVerticle`** (extension) is the scan as an *active* component. The extension needs state that lives during the scan and changes over time: counters, a stopped flag, a periodic timer, a subscription to stop messages. Putting this state in a verticle confines it to one event loop, so plain fields are enough. One verticle is deployed for each scan and it undeploys itself at the end: a new scan never shares state with an old one, and the lifecycle of the scan coincides with the lifecycle of the verticle.
- **`VertxController`** translates GUI commands into Vert.x actions: *start* = deploy a new `FSStatVerticle`, *stop* = publish on `fsstat.stop`. It deliberately keeps no reference to the running verticle (not even its deployment id, which would be written by an event loop and read by the EDT): the only link between the EDT and the scan is message passing.
- **`ViewAgent`** receives the messages of the scan and forwards them to the view. It is a verticle, rather than a set of consumers registered by `main`, for the same reason as `ReportAgent`: by construction, the handlers of `fsstat.update` and `fsstat.done` run on one known event loop, one at a time, and messages sent by the scanning verticle are handled in the order they were sent, so a late update cannot overwrite the final report. It also follows the lab examples (event-bus consumers registered in a verticle), and it is deployed before the window is shown, so the consumers exist before the first scan can start. `ViewAgent` is the bridge between the event-loop world and the EDT.

Figure 6 shows the interactive use case: the controller deploys a new `FSStatVerticle`, which publishes updates every 100 ms; a stop request is a message, after which the verticle publishes the final outcome and undeploys itself.

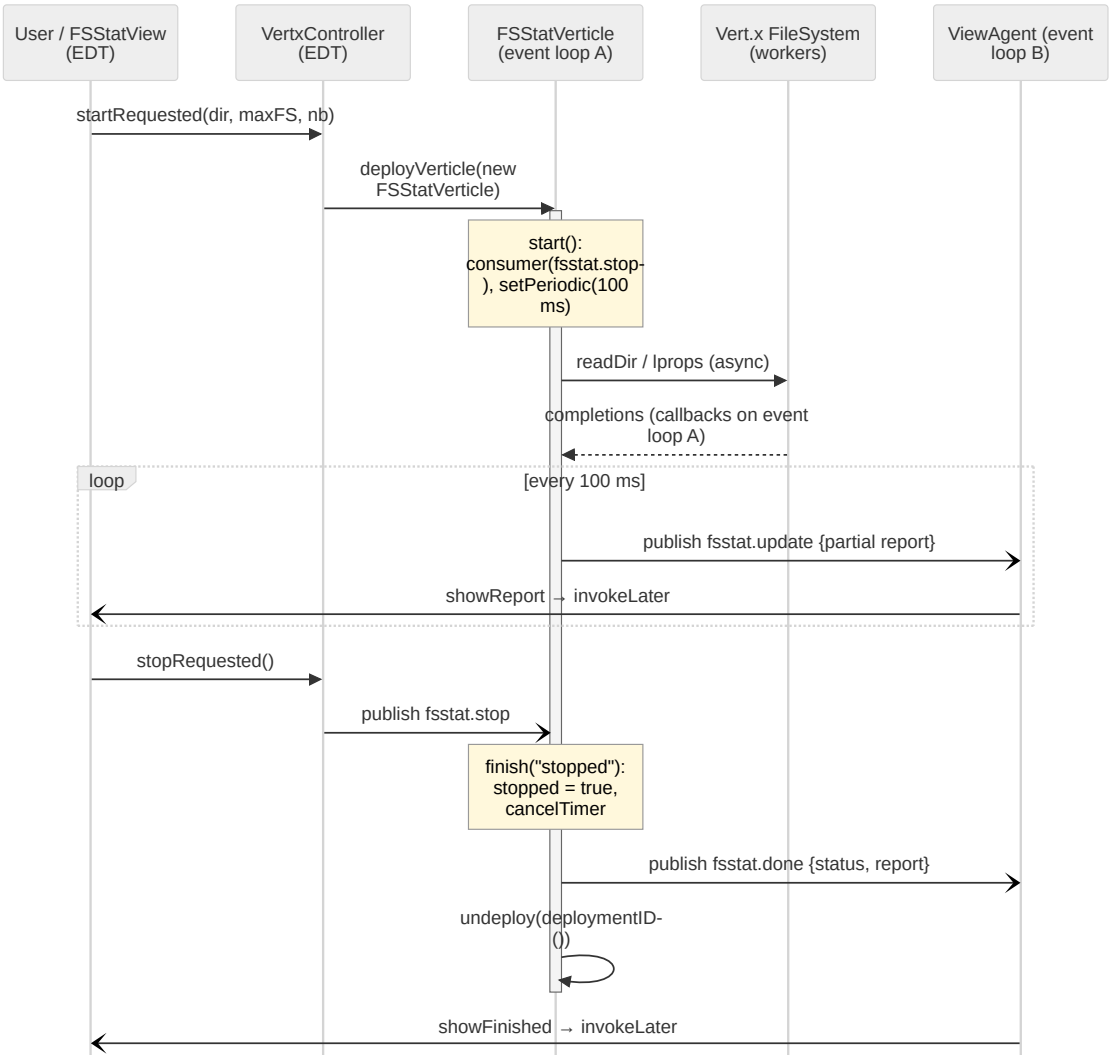


Figure 6: Event-loop version: sequence diagram of start, updates and stop.

2.3 Reactive version: components

The reactive version has the smallest structure (Figure 7): a single library class exposing two cold streams, and a controller that owns the `Disposable` of the running scan.

In Rx the unit of design is the *stream*, and concurrency is a property of the pipeline (schedulers), not of the components. The library therefore only *describes* computations; they run when someone subscribes, and stop when the subscription is disposed.

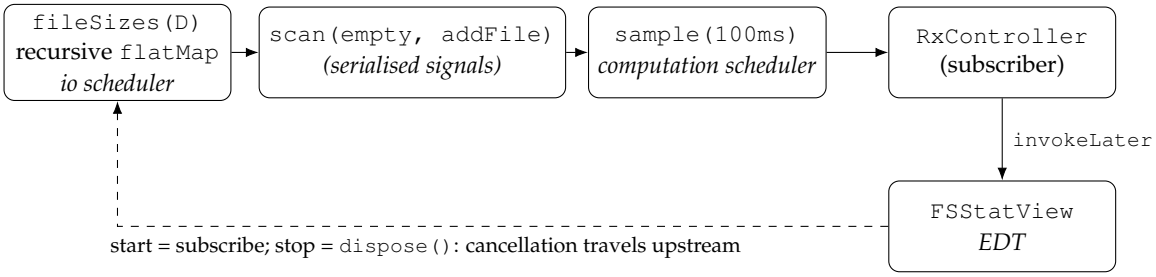


Figure 8: Reactive version, GUI program: the interactive pipeline.

- **`FSStatLibRx`** is a factory of cold streams with no threads and no state of its own. Its core

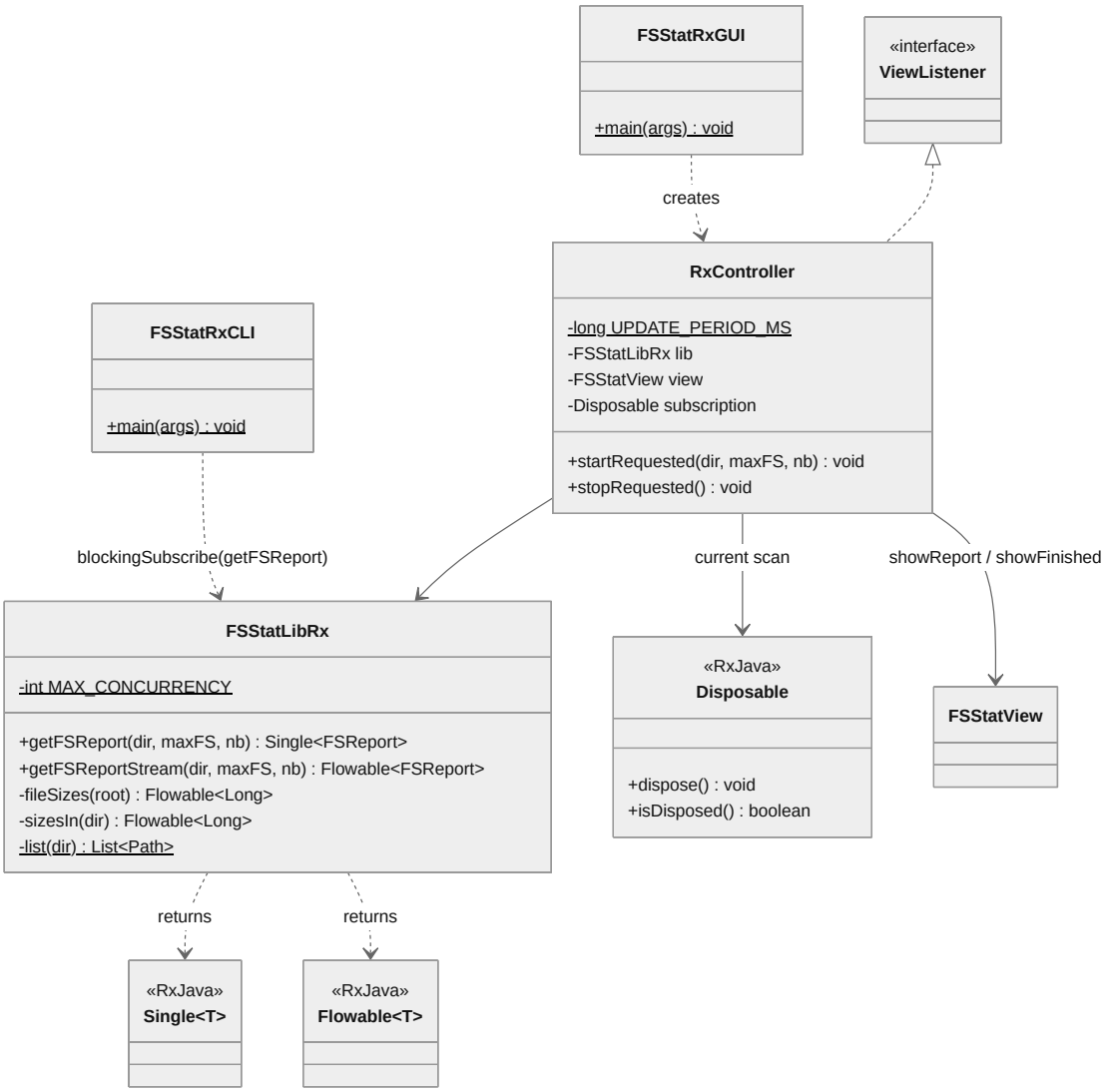


Figure 7: Reactive version: class diagram.

is `fileSizes(D)`, the stream of the sizes of all the regular files under `D`; the two public methods are folds over it: `reduce` gives the final report (`Single`), `scan` the sequence of partial reports (`Flowable`). The base method and the extension are thus two views of the same stream, which is the reactive way to satisfy R6.

- **No agents are needed**: the role played by `FSStatVerticle` in Vert.x (owning the partial state) is played by the `scan` operator, whose accumulator is safe because Rx serialises the signals of a stream; the role of the periodic timer is played by `sample`, chosen by the *consumer*, which is where the desired update rate is known.
- **`RxController`** is the subscriber. It owns the `Disposable` of the current scan, used only by the EDT (start and stop are both GUI events), so it needs no synchronization. Errors and completion arrive as stream signals.
- **`FSStatRxCLI`** simply subscribes to the `Single` with `blockingSubscribe`, as in the lab examples, to keep `main` alive until the end.

In the sequence diagram (Figure 9) note that calling `getFSReportStream` does not start anything: the scan begins at `subscribe` and ends either with `onComplete` or with `dispose()`.

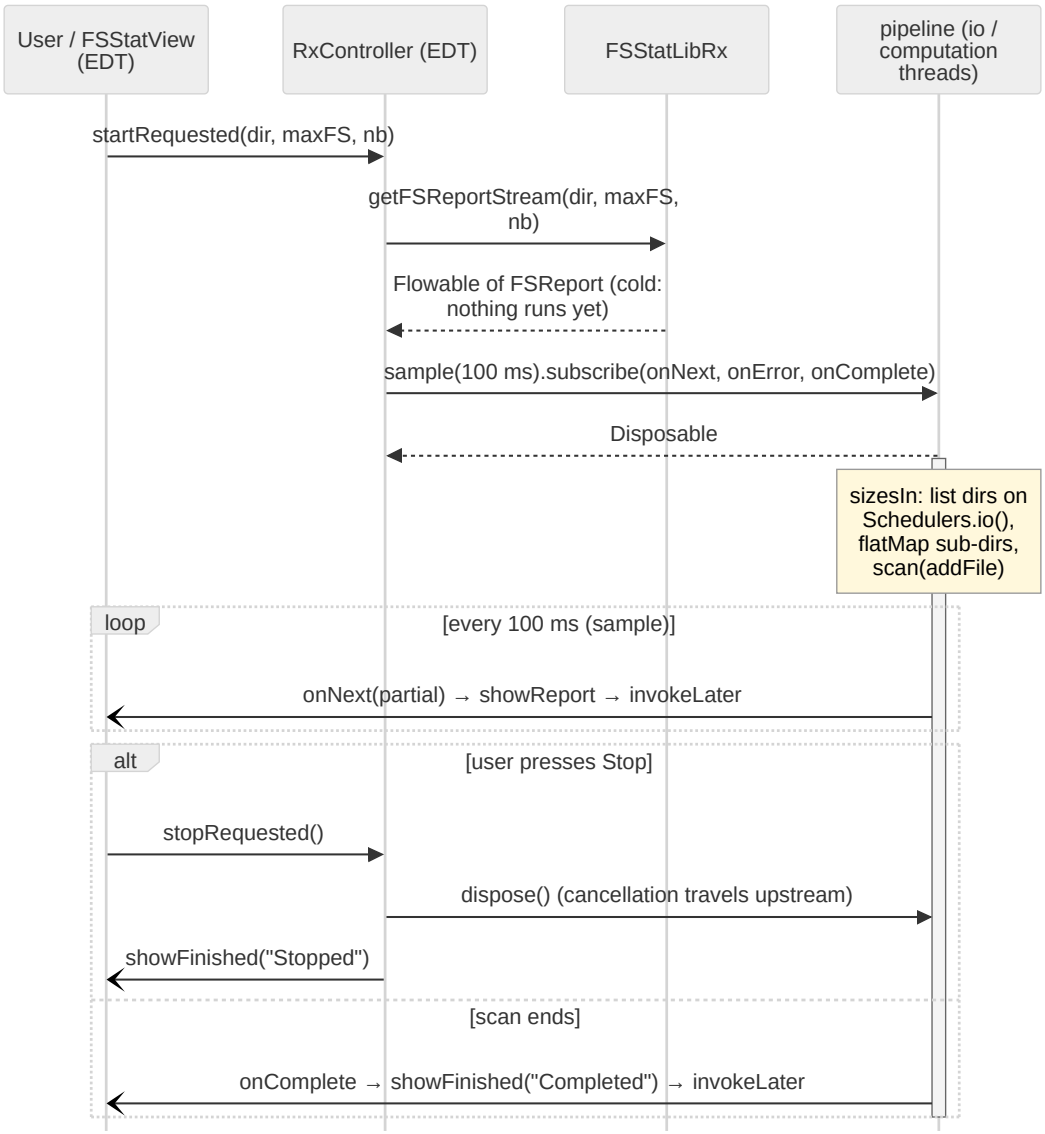


Figure 9: Reactive version: sequence diagram of start, updates and stop.

2.4 Virtual-threads version: components

As for Vert.x, the classes are split between the base library (Figure 10) and the interactive extension (Figure 11). The extension follows the active/passive scheme of the course: `FSStatTask` owns the threads, `StatsMonitor` and `Flag` are the monitors they share, `FSStatListener` is the observer interface.

With virtual threads the unit of concurrency is again the thread, but threads are cheap enough to create one per sub-directory and to block freely. The design is therefore the classic one of the course: active entities (threads) plus passive shared entities (monitors).

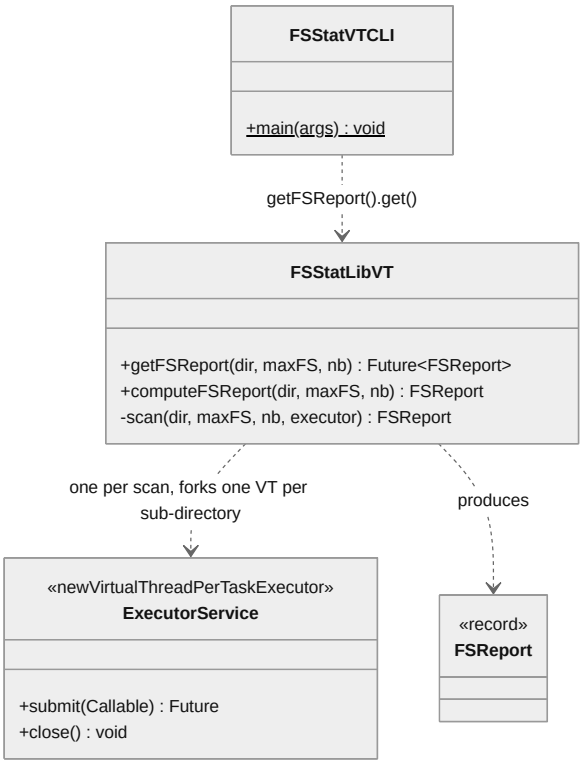


Figure 10: Virtual-threads version, base library and CLI example (class diagram).

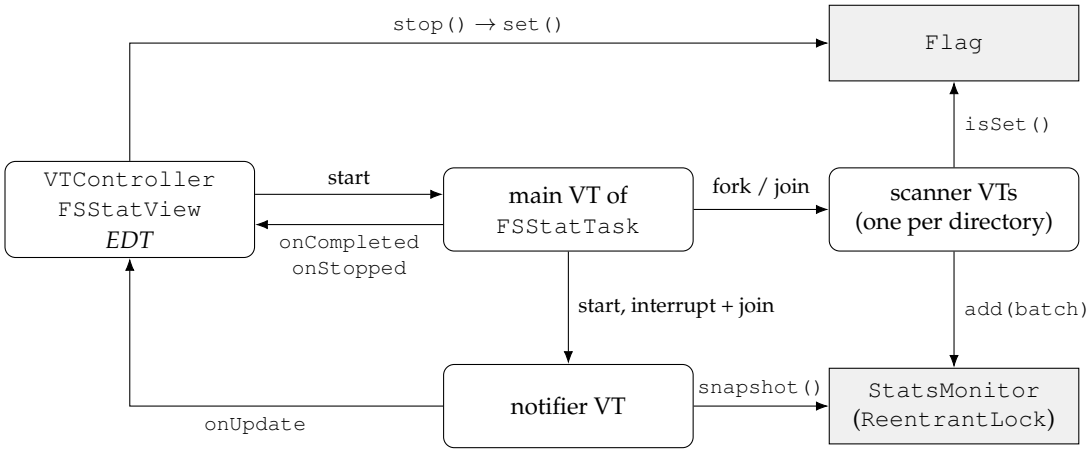


Figure 12: Virtual-threads version, GUI program. Grey boxes are monitors; the listener calls end in `invokeLater` on the view.

- **`FSStatLibVT`** (base) is written as sequential blocking code. One virtual thread per sub-directory realises the divide and conquer of the analysis: each thread counts its own files in a local array and returns an immutable report through a `Future`, so the base version shares nothing. `getFSReport` runs the computation on a new virtual thread and returns a `Future`, which is what makes the call asynchronous (R1).
- **`FSStatTask`** (extension) is an *active object* representing a scan: its main virtual thread runs the fork/join scan, starts the notifier and, at the end, decides the outcome and calls the listener. One instance per scan, as for the verticle in Vert.x.
- **`StatsMonitor`** is the shared partial result that R6 requires. Writers are the scanner threads (in batches of 256 files, to keep contention low), the reader is the notifier. It is guarded by

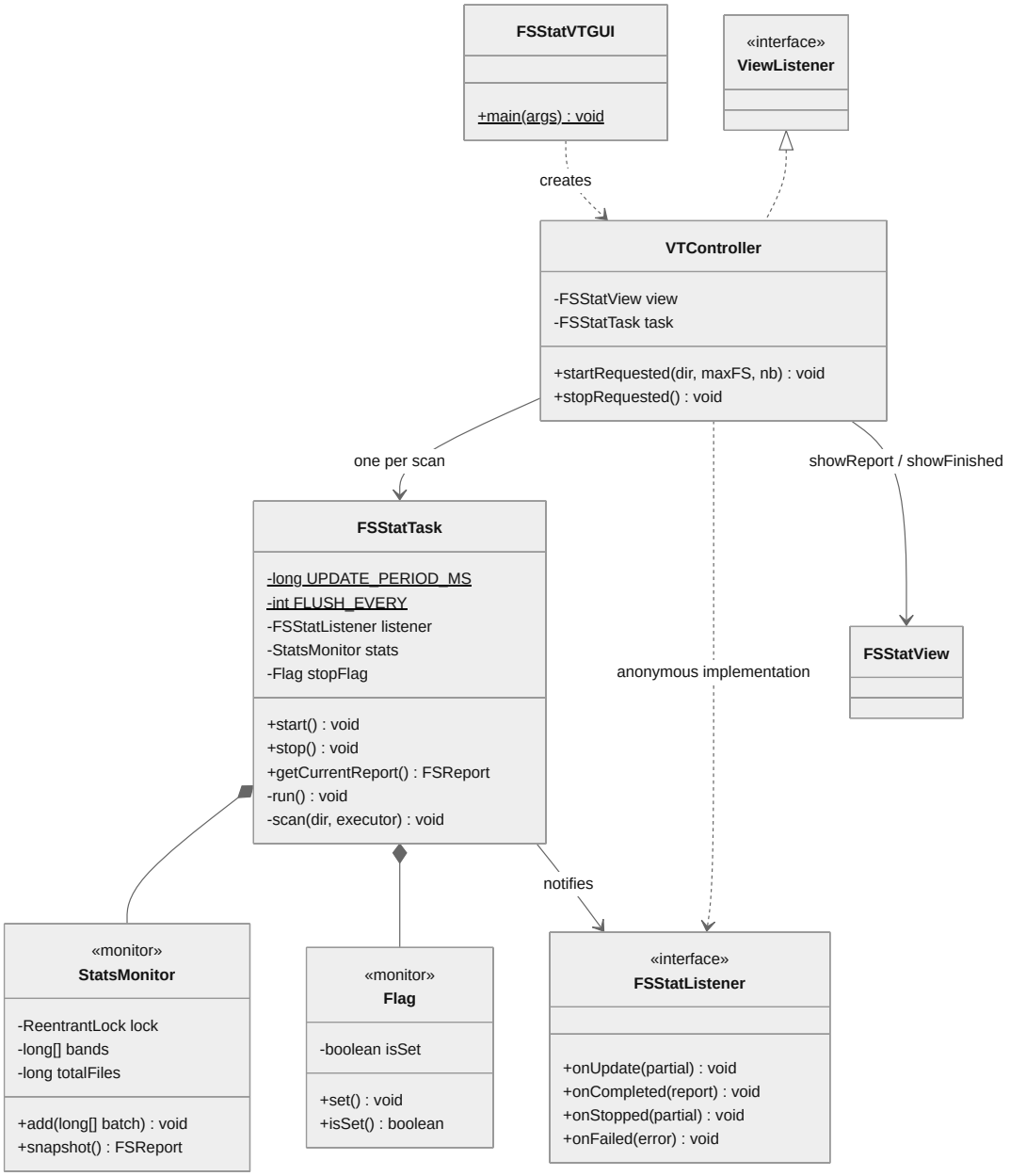


Figure 11: Virtual-threads version, interactive extension and GUI program (class diagram).

a `ReentrantLock`: on Java 21 a virtual thread that blocks on a `synchronized` monitor pins its carrier thread, while blocking on a `java.util.concurrent` lock simply parks the virtual thread.

- **The notifier** is a separate virtual thread so that the update rate (every 100 ms) does not depend on the scanning threads: they only write, it only reads and pushes.
- **`Flag`** is the stop signal, the same monitor used in the course examples (`chrono_mvc`). Cancellation is cooperative: each scanner checks it before every entry. Its critical sections are tiny and never block, so `synchronized` is harmless here.
- **`FSStatListener`** is the observer interface (as `CounterEventListener` in the lab): its methods are called by virtual threads, so `VTController` implements them by calling the thread-safe view methods.

Figure 13 shows the interactive use case: the scanner threads write into the monitor in batches,

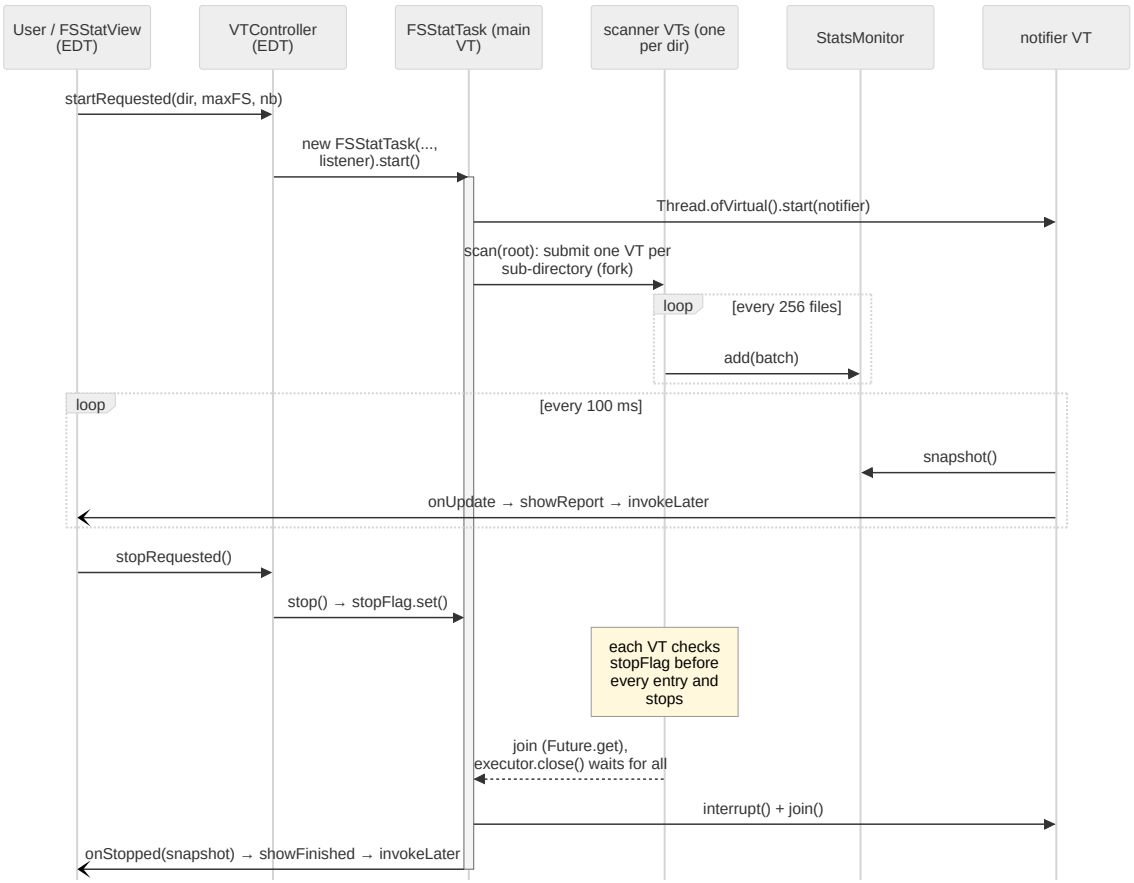


Figure 13: Virtual-threads version: sequence diagram of start, updates and stop.

the notifier reads it periodically, and a stop sets the flag; the main thread of the task waits for the scanners and for the notifier before the final notification.

2.5 Cross-cutting decisions

	Event loop	**Reactive**	**Virtual threads**
Partial state (R6)	counters in `FSStatVerticle`, confined to its event loop	`scan` accumulator, serialised by Rx	`StatsMonitor`, lock-protected
Who limits the rate	periodic timer of the verticle	`sample`, chosen by the subscriber	notifier virtual thread
Stop (R5)	message on `fsstat.stop`; `stopped` flag checked by callbacks; undeploy	`dispose()`: cancellation upstream	`Flag` checked before every entry
No update after the end	timer cancelled before publishing `done`; one event loop handles both addresses	Rx contract: nothing after a terminal signal or after `dispose`	notifier interrupted and joined before the final callback
Restart	new verticle per scan	new subscription per scan	new `FSStatTask` per scan

Table 3: How each version addresses the concurrency aspects of the extension.

In all versions the GUI is updated by *pushing* reports, but the library never touches Swing and never decides how the view is drawn: it only produces reports at a bounded rate. The view methods are the single point where control passes to the EDT, and the EDT never blocks on the library: it only deploys/publishes, subscribes/disposes, or starts/stops a task.

3 Event-loop version (Vert.x)

3.1 Base: map-reduce by composing futures

`FSStatLibVertx.getFSReport` returns a `Future<FSReport>` and is meant to be called from a verticle (as in the example `FSStatVertxCLI`). The recursion is a *non-blocking* map-reduce: each entry becomes a future (a file becomes a one-file report, a sub-directory a recursive scan) and `Future.all` joins them; the merge runs in the continuation.

```
private Future<FSReport> scanDir(String dir, FSReport empty) {
  return fs.readDir(dir).compose(entries -> {
    List<Future<FSReport>> children = entries.stream()
        .distinct().filter(e -> isChildOf(e, dir))     // drops resolved symbolic links
        .map(e -> scanEntry(e, empty)).toList();
    return Future.all(children).map(all -> {           // reduce
      FSReport report = empty;
      for (FSReport child : all.<FSReport>list()) report = report.merge(child);
      return report;
    });
  });
}
private Future<FSReport> scanEntry(String path, FSReport empty) {
  return fs.lprops(path).compose(p -> p.isDirectory() ? scanDir(path, empty)
        : Future.succeededFuture(p.isRegularFile() ? empty.addFile(p.size()) : empty))
      .otherwise(empty);                               // unreadable entry: counts nothing
}
```

Termination needs no counter: the future of a directory completes exactly when the futures of all its entries have completed, so the root future completes when the whole tree is done. **Threads**: every continuation runs on the event loop of the calling verticle, one at a time, so there is no shared mutable state and no synchronization. The blocking file-system calls are executed by Vert.x on its internal worker pool, and their results come back to the event loop as events: the event loop itself never blocks. Two details: `readDir` returns *canonical* paths (links already resolved), so a link is recognised as an entry that is not a direct child of the directory (or a duplicate) and is discarded; for the same reason the root is canonicalised once with `executeBlocking`, the standard way to run a short blocking call.

3.2 Extension: the scan as a verticle

`FSStatVerticle` is an event loop with its own state (counters, a `stopped` flag, a timer): it scans with the same future-based recursion, incrementing its counters directly, and interacts with the rest of the application only through the event bus (Figure 5):

- every 100 ms a periodic timer publishes the current report on `fsstat.update`;
- when the scan ends, or a message arrives on `fsstat.stop`, `finish(status)` sets `stopped`, cancels the timer, publishes the outcome (`completed`, `stopped` or `failed`) on `fsstat.done` and undeploys the verticle;
- pending callbacks check `stopped` and return immediately, so the remaining futures complete quickly and no update can follow the final message.

Since callbacks, timer and event-bus handlers are all run by the verticle's event loop, the flag and

the counters are plain fields: there is no race between the scan, the stop request and the updates. Every scan is a new verticle instance, so a new scan never shares state with an old one; on the GUI side `ViewAgent` handles updates and the final outcome in order and passes them to the EDT.

4 Reactive version (RxJava)

The file system is modelled as a cold stream of file sizes; the report is a fold over it.

```
public Single<FSReport> getFSReport(Path dir, long maxFS, int nb) {
  return Single.defer(() -> fileSizes(dir)
      .reduce(FSReport.empty(maxFS, nb), FSReport::addFile));
}
public Flowable<FSReport> getFSReportStream(Path dir, long maxFS, int nb) {
  return Flowable.defer(() -> fileSizes(dir)
      .scan(FSReport.empty(maxFS, nb), FSReport::addFile));
}
private Flowable<Long> sizesIn(Path dir) {
  return Flowable.defer(() -> Flowable.fromIterable(list(dir)))
      .subscribeOn(Schedulers.io())                    // blocking listing on an I/O thread
      .flatMap(entry -> Flowable.defer(() -> {
          var attrs = Files.readAttributes(entry, BasicFileAttributes.class, NOFOLLOW_LINKS);
          if (attrs.isDirectory())   return sizesIn(entry);   // recursion: a sub-stream
          if (attrs.isRegularFile()) return Flowable.just(attrs.size());
          return Flowable.<Long>empty();
        }).onErrorResumeNext(err -> Flowable.empty()),  // unreadable entry: skipped
        MAX_CONCURRENCY);
}
```

- **Concurrency.** Each directory listing is subscribed on `Schedulers.io()`, so sub-trees are explored in parallel and `flatMap` merges their streams. Its `maxConcurrency` argument (8) bounds the sub-streams active *per directory*; since the bound applies at each level of the recursion, the overall concurrency is not globally limited to 8 (the I/O scheduler grows as needed).
- **Termination** is part of the protocol: `flatMap` completes when the source and all inner streams have completed, so the stream of a directory completes after its whole sub-tree.
- **Accumulation without shared state.** `reduce` (base) and `scan` (extension) fold the sizes into immutable reports. Even though items come from different threads, Rx guarantees that the `onNext` signals merged by `flatMap` are serialised, so the accumulator is never accessed concurrently: no locks, no atomics.
- **Errors.** A failure on a single entry is recovered locally (`onErrorResumeNext`); invalid parameters or a root that is not a directory travel to the subscriber as an `onError` signal.
- **Updates and backpressure.** The interactive stream emits one partial report per file; the consumer decides the rate with `sample(100ms, emitLast=true)`, which keeps the final report. **Stop** is `dispose()`: cancellation travels upstream through every operator, and nothing is emitted afterwards.

Nothing is computed until a subscription is made: the example `FSStatRxCLI` uses `blockingSubscribe` on the `Single`, the GUI controller subscribes to the stream and keeps the `Disposable`.

5 Virtual-threads version

5.1 Base: fork/join with blocking, sequential code

The code is plain blocking Java. A virtual thread scans one directory: it counts its files in a *local* array, forks one virtual thread per sub-directory (`executor.submit`) and then joins them with `Future.get()`, merging the returned immutable reports.

```
private FSReport scan(Path dir, long maxFS, int nb, ExecutorService executor)
    throws Exception {
  long[] bands = new long[nb + 1];                    // local to this virtual thread
  List<Future<FSReport>> subDirs = new ArrayList<>();
  try (DirectoryStream<Path> entries = Files.newDirectoryStream(dir)) {
    for (Path entry : entries) {
      var attrs = Files.readAttributes(entry, BasicFileAttributes.class, NOFOLLOW_LINKS);
      if (attrs.isDirectory())                                    // fork
        subDirs.add(executor.submit(() -> scan(entry, maxFS, nb, executor)));
      else if (attrs.isRegularFile())
        bands[FSReport.bandOf(attrs.size(), maxFS, nb)]++;
    }                                                 // (IOException handling omitted)
  }
  FSReport report = FSReport.of(maxFS, bands);
  for (Future<FSReport> f : subDirs) report = report.merge(f.get());   // join
  return report;
}
```

Blocking on `get()` or on I/O is cheap because a blocked virtual thread is unmounted and releases its carrier thread; thousands of directories can be in progress at the same time without dedicating an OS thread to each one. Termination is the natural end of the recursive joins, and the executor (`newVirtualThreadPerTaskExecutor`, closed with try-with-resources) also waits for all the forked threads. Results flow back through the futures, so the base version shares *no* mutable state. `getFSReport` runs `computeFSReport` on a new virtual thread and returns a `Future`; a caller that is itself a virtual thread can simply call the synchronous version.

5.2 Extension: shared monitor, notifier, stop flag

To make partial results observable, `FSStatTask` keeps the same fork/join structure but each virtual thread periodically flushes its local counters (every 256 files and at the end of the directory) into a shared monitor, `StatsMonitor`. A dedicated *notifier* virtual thread reads a snapshot every 100 ms and pushes it to the listener.

- `StatsMonitor` protects its counters with a `ReentrantLock` rather than `synchronized`: on Java 21 a virtual thread blocked while entering a `synchronized` block pins its carrier thread, while blocking on a `java.util.concurrent` lock simply parks it. Batching keeps contention on the lock low.
- Stop is cooperative: `stop()` sets a `Flag` (a tiny monitor) checked by every virtual thread before each entry; the threads stop forking, the joins complete quickly, and the main virtual thread of the task notices the flag and calls `onStopped`.
- Before calling `onCompleted` or `onStopped`, the main virtual thread of the task interrupts and *joins* the notifier, so no update can arrive after the final notification.

6 GUI and MVC

The view (Figure 14) has fields for `D`, `NB` and `MaxFS`, Start/Stop buttons, a histogram and a text area with the band ranges. It validates the input, disables Start while a scan runs and re-enables it when the controller reports the outcome. Each version has its own controller (`VertxController`,

`RxController`, `VTController`) which only translates commands into the primitives of its library: deploy/publish a message, subscribe/dispose, start/stop a task. The `Disposable` and the `FSStatTask` are only accessed by the EDT.

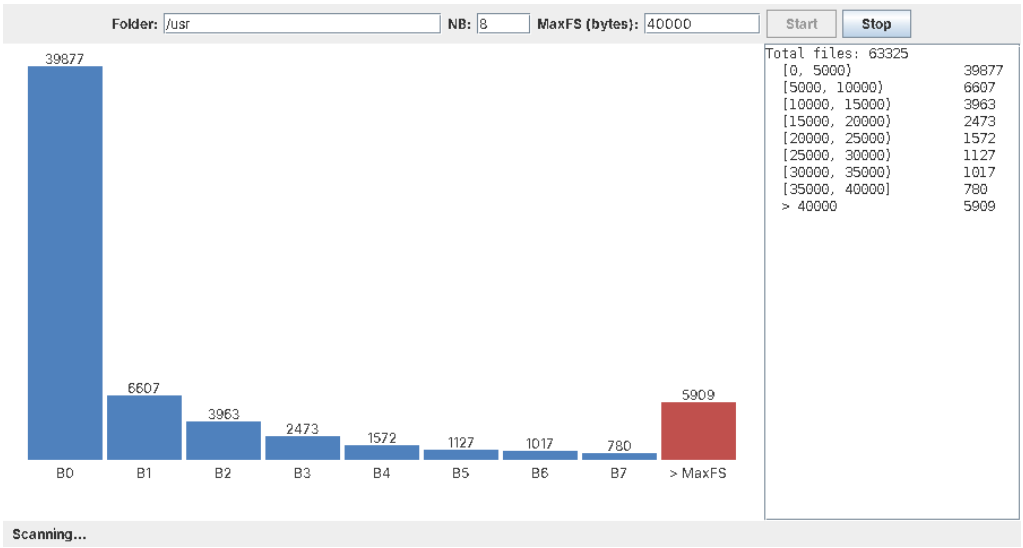


Figure 14: The GUI (virtual-threads version) while scanning `/usr` with $NB = 8$, $MaxFS = 40000$.

7 Comparison and verification

	Event loop	**Reactive**	**Virtual threads**
Style	continuations on futures	declarative pipeline	imperative, blocking
Who waits for I/O	Vert.x worker pool; event loop never blocks	I/O scheduler threads	the virtual thread (unmounted)
Termination	`Future.all` composition	`flatMap` completion	recursive joins
Aggregation	merge in continuations (base), confined counters (ext.)	`reduce` / `scan`	local arrays + futures (base), monitor (ext.)
Shared state	none (thread confinement)	none (serialised signals)	none (base), lock-protected monitor (ext.)
Updates	periodic timer, event bus	stream + `sample`	notifier virtual thread
Stop	message, `stopped` flag, undeploy	`dispose()`	cooperative `Flag`

Table 4: The three solutions side by side.

The three versions were checked against an independent count (a Python script) on a synthetic tree with files on the band boundaries (0, $MaxFS/NB$, $MaxFS$, $MaxFS + 1$ bytes) and a symbolic link creating a cycle, and on `/usr` (134 929 files): all of them return exactly the same report. Further checks: an unreadable sub-directory is skipped (run as a non-root user) and a FIFO is ignored; a non-existent root, a file used as root and invalid parameters produce an error; in the interactive versions stop, restart and completion were exercised repeatedly, verifying that no update is delivered after the final outcome and that the scanning verticle undeploys itself.